📚 DAY 3: MASTER TERMINOLOGY & CONCEPT GLOSSARY

Deep Learning on HPC with TensorFlow, Slurm & GPUs

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 3 (Strictly 3 Hours / 180 Minutes)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

📌 Master Vocabulary & Deep Learning HPC Mechanics (Terms #1 – #36)

1. Hardware-Aware Deep Learning Optimization

- **Definition**: The engineering practice of tailoring neural network computational graphs, tensor precision, and memory layouts to the microarchitecture of the physical silicon (CPU vector units, GPU systolic arrays, or dedicated tensor accelerators).
- **HPC Impact**: Bypasses generic interpreter overhead, maximizing execution unit occupancy and eliminating memory bandwidth stalls.

2. Intel oneDNN (oneAPI Deep Neural Network Library)

- **Definition**: An open-source cross-architecture performance library providing fundamental computational primitives for deep learning applications.
- **Key Operations**: Highly vectorized 2D/3D Convolutions, Multi-Head Attention (MHA), Matrix Multiplication (GEMM), Pooling, Layer Normalization, Batch Normalization, and fused Activation functions (GELU, ReLU).
- **Silicon Target**: Direct microarchitectural dispatch to Intel AVX-512 and Intel AMX on Xeon CPUs, and Intel XMX systolic arrays on GPUs.

3. TensorFlow-MKL (`tensorflow-mkl`)

- **Definition**: A distribution of TensorFlow compiled with Intel oneDNN and Math Kernel Library primitives enabled by default.
- **Environment Variable**: `TF_ENABLE_ONEDNN_OPTS=1`.
- **Mechanism**: Automatically substitutes generic Eigen C++ graph operations with microarchitecture-optimized oneDNN kernels.

4. Operator Fusion (Kernel Fusion)

- **Definition**: Combining multiple contiguous neural network layers (e.g., Convolution + Bias Add + ReLU activation) into a **single fused computational kernel**.
- **HPC Silicon Benefit**: Eliminates intermediate memory round-trips. Instead of writing the convolution output back to DRAM and reading it again for ReLU, the activation function evaluates directly while intermediate results are still hot in CPU L1 cache or GPU registers.

5. Intel AMX (Advanced Matrix Extensions)

- **Definition**: A hardware accelerator block introduced in 4th/5th Generation Intel Xeon Scalable processors (Sapphire Rapids / Emerald Rapids) dedicated to matrix math.
- **Silicon Architecture**:
  - **Tile Registers (`tmm0` – `tmm7`)**: 8 two-dimensional registers holding 1 KB of data each (8 KB total state).
  - **TMUL (Tile Matrix Multiply Unit)**: A systolic execution unit that computes 1024 BF16 operations or 2048 INT8 operations per clock cycle.

6. Habana Gaudi (Gaudi 2 & Gaudi 3) AI Accelerator

- **Definition**: A purpose-built, high-throughput ASIC architecture engineered from the ground up for deep learning training and massive generative AI workloads.
- **Key Engines**:
  - **MME (Matrix Multiplication Engine)**: Dedicated 3D GEMM and 2D convolution systolic arrays.
  - **TPC (Tensor Processing Cores)**: Fully programmable VLIW SIMD processors executing non-linear operations (Softmax, GELU, RoPE).

7. Habana SynapseAI Software Stack

- **Definition**: The complete software suite, runtime, and graph compiler connecting high-level frameworks (PyTorch, TensorFlow) to Habana Gaudi hardware.
- **Mechanism**: Compiles computational graphs into optimized hardware microcode, fusing operators and allocating memory buffers directly in Gaudi HBM.

8. Slurm Workload Manager

- **Definition**: The industry-standard cluster management and job scheduling system for Linux supercomputers.
- **Core Responsibilities**:
  1. Allocates exclusive and/or non-exclusive access to compute nodes.
  2. Provides a framework for starting, executing, and monitoring parallel jobs.
  3. Arbitrates resource contention by managing a queue of pending work.

9. Slurm Architecture: `slurmctld` vs. `slurmd`

- `slurmctld` **(Slurm Controller Daemon)**: Runs on the supercomputer Master/Head Node; monitors cluster state, evaluates job resource requirements, and schedules pending jobs.
- `slurmd` **(Slurm Node Daemon)**: Runs on every individual Compute Node; executes tasks assigned by `slurmctld`, tracks local hardware state, and returns exit codes.

10. `sbatch` (Slurm Batch Submission)

- **Definition**: The command used to submit a non-interactive batch job script to the Slurm queue. The script contains `#SBATCH` directives requesting hardware resources.

11. `squeue` & `sinfo`

- `sinfo`: Displays the health, state, and partition configurations of all cluster nodes (e.g., `idle`, `alloc`, `down`).
- `squeue`: Displays active, queued, and running jobs currently managed by the scheduler.

12. Slurm Directives: Resource Allocation

- `#SBATCH --nodes=N`: Requests N physical compute servers.
- `#SBATCH --ntasks-per-node=T`: Launches T independent MPI tasks/processes per server.
- `#SBATCH --cpus-per-task=C`: Assigns C CPU cores to each task (for OpenMP multithreading).
- `#SBATCH --gres=gpu:G`: Requests G hardware accelerators/GPUs per node.
- `#SBATCH --time=HH:MM:SS`: Enforces maximum job wall-clock time limit.

13. Data Parallelism vs. Model Parallelism

- **Data Parallelism**: The entire neural network model is duplicated across all worker nodes. Each worker processes a distinct mini-batch slice of data, and gradients are averaged across workers during backpropagation.
- **Model Parallelism**: The model is too massive to fit on a single device (e.g., 70B parameter LLM); individual layers or tensor dimensions are split across multiple GPUs/accelerators.

14. Ring-AllReduce Algorithm

- **Definition**: A bandwidth-optimal collective communication algorithm that aggregates gradients across N workers without a centralized master bottleneck.
- **Network Flow**:
  1. Workers are arranged in a logical ring ($0 \to 1 \to 2 \to \cdots \to N-1 \to 0$).
  2. **Scatter-Reduce Phase**: Each worker sends $\frac{1}{N}$ of its tensor to its neighbor and receives a slice from its predecessor for $N-1$ steps.
  3. **AllGather Phase**: The fully reduced slices are circulated around the ring for another $N-1$ steps until all workers possess the identical aggregated gradient.
- **Communication Volume**: $2 \times \frac{N-1}{N} \times \text{Size}$, which is completely independent of the number of nodes N for large models!

15. Horovod Distributed Framework

- **Definition**: An open-source distributed deep learning training framework created by Uber that wraps TensorFlow, PyTorch, and Keras with Ring-AllReduce communication.
- **Key Superpower**: Replaces standard parameter-server models with high-speed MPI/NCCL/oneCCL primitives, requiring only 4–5 lines of code modifications.

16. `hvd.init()`

- **Definition**: Initializes the Horovod distributed runtime, establishing MPI communicators, process ranks, and hardware binding.

17. `hvd.local_rank()` vs. `hvd.rank()`

- `hvd.rank()`: The global worker ID across the entire multi-node cluster (0 to $\text{WorldSize} - 1$).
- `hvd.local_rank()`: The local device ID within the current physical node (0 to $G - 1$). Used to bind the process to a specific local GPU via `tf.config.set_visible_devices()`.

18. `hvd.DistributedOptimizer()`

- **Definition**: A wrapper around any standard optimizer (e.g., Adam, SGD) that intercepts backpropagation gradients and executes asynchronous Ring-AllReduce across all workers before updating weights.

19. `hvd.broadcast_variables()`

- **Definition**: Broadcasts the master worker's (Rank 0) initial model weights and optimizer state to all other ranks, ensuring all workers start training from the exact same mathematical state.

20. Learning Rate Warmup & Linear Scaling Rule

- **Rule**: When training with K data-parallel workers, the effective global batch size is $K \times \text{LocalBatchSize}$. Therefore, the base learning rate should be scaled linearly: $\text{LR}_{\text{global}} = K \times \text{LR}_{\text{base}}$.
- **Warmup**: Gradually ramping up the learning rate over the first 5 epochs prevents gradient explosions early in training.

21. Mixed Precision Training (FP16 / BF16)

- **Definition**: Executing forward and backward passes using 16-bit floating-point numbers (FP16 or Bfloat16) while maintaining weights in 32-bit (FP32) master copies.
- **Benefit**: Halves memory footprint and doubles tensor execution throughput on hardware systolic arrays (AMX, XMX, Tensor Cores).

22. Bfloat16 (Brain Floating Point)

- **Definition**: A 16-bit floating point format with an 8-bit exponent and 7-bit mantissa, matching the dynamic range of standard FP32.
- **Advantage**: Eliminates loss scaling requirements needed by standard IEEE FP16, preventing gradient underflow in deep networks.

23. Computational Graph (Static vs. Dynamic)

- **Static Graph (`tf.function`)**: The model structure is compiled into an optimized execution graph before runtime, allowing oneDNN operator fusion and dead-code elimination.
- **Dynamic Graph (Eager Execution)**: Operations are evaluated line-by-line as called in Python.

24. Input Pipeline Bottleneck (I/O Bound ML)

- **Definition**: A condition where high-speed GPUs sit idle at 10% utilization because the CPU storage pipeline cannot decode and augment image/audio batches fast enough.
- **Fix**: Use `tf.data.Dataset` with prefetching (`dataset.prefetch(tf.data.AUTOTUNE)`), multi-threaded map (`num_parallel_calls`), and high-speed `/scratch` NVMe storage.

25. Checkpointing in HPC Deep Learning

- **Definition**: Periodically saving model weights and optimizer states to persistent storage during multi-hour cluster runs.
- **HPC Rule**: Only **Rank 0** should write checkpoints to `/scratch` to prevent 100 workers from concurrently attempting to overwrite the same file over the network.

26. Scaling Efficiency in Distributed DL

- **Formula**: $\text{Efficiency} = \frac{\text{Throughput on } K \text{ workers}}{K \times \text{Throughput on 1 worker}} \times 100\%$.
- **Benchmark Target**: Production HPC clusters aim for $\geq 85\%$ scaling efficiency on 64+ accelerators.

27. Weak Scaling vs. Strong Scaling

- **Strong Scaling**: Global batch size remains fixed while increasing worker count K. (Execution time per epoch should decrease).

- **Weak Scaling**: Batch size per worker remains fixed, so global batch size scales as $K \times B$. (Execution time should remain constant as problem size grows).

28. Gradient Accumulation

- **Definition**: Accumulating gradients over multiple sequential forward/backward mini-batches before calling `optimizer.step()`.
- **Use Case**: Simulates massive batch sizes (e.g., 4096) when GPU VRAM limits local batch sizes to 16.

29. Asynchronous vs. Synchronous AllReduce

- **Synchronous**: Every worker waits for all other workers to finish their mini-batch before exchanging gradients (ensures strict mathematical equivalence).
- **Asynchronous**: Workers update a global parameter server without waiting (faster, but suffers from stale gradients and unstable convergence).

30. Interconnect Fabric: InfiniBand vs. Ethernet

- **Standard 1 GbE/10 GbE**: High TCP latency (10–30 μs), causes AllReduce communication stalls.
- **InfiniBand HDR/NDR & RoCE v2**: RDMA (Remote Direct Memory Access) bypasses the CPU kernel, achieving sub-microsecond latency (< 1 μs) and 200–400 Gbps bandwidth.

31. Straggler Node Effect

- **Definition**: In synchronous distributed training, the entire cluster must wait at the AllReduce barrier for the slowest compute node. A single thermal-throttling GPU can slow down 128 nodes!

32. Weight Synchronization Drift

- **Definition**: When numerical rounding errors in floating-point math cause model weights across data-parallel workers to diverge over millions of steps if not strictly synchronized.

33. `tf.config.optimizer.set_jit(True)`

- **Definition**: Enables XLA (Accelerated Linear Algebra) compiler optimization in TensorFlow, fusing graph operations into optimized machine assembly.

34. Throughput Metric: Images per Second

- **Equation**: $\text{Throughput} = \frac{\text{Batch Size} \times \text{Steps}}{\text{Elapsed Time (Seconds)}}$.

35. Intel Xe Link Bridges

- **Definition**: High-speed, coherent hardware interconnect bridges providing direct point-to-point communication between Intel Data Center GPU Max tiles, bypassing the PCIe host bus.

36. Ephemeral Scratch vs. Persistent Home in DL

- **Rule**: Raw datasets (ImageNet, Parquet) and training checkpoints must sit in high-bandwidth parallel storage (`/scratch`), never in the user's quota-constrained network home (`/home`).

Glossary Status: 100% Complete & Verified | GLA University HPC Masterclass Day 3